

Weekly Project Progress Report

Team Name: Quick Bite

Week Number: Week 9 – Database Schema Design

Team Members:

1. Abdullah Mohammed AbuZaid	120211559
2. Abdullah Ali Al-Hindawi	120211863
3. Abdalkareem Rajab Abo Younis	120211877
4. Hazem Mohamed Oukal	120212771

GitHub Repository: <https://github.com/AbdullahAbuZaid04/quick-bite-restaurant.git>

Work completed this week

- Designed and implemented the full MySQL relational database schema for the Quick Bite food ordering system.
- Created seven core tables: users, categories, menu_items, orders, order_items, payments, and invoices.
- Defined all primary keys, foreign key constraints, and unique indexes to enforce data integrity.
- Authored two automated triggers to calculate order item subtotals and update the parent order's total_amount automatically on insert.
- Configured the database with UTF-8 (utf8mb4) character encoding for full Arabic and multilingual support.
- Seeded the database with realistic Palestinian sample data including menu items, users, and orders from Gaza.
- Pushed the complete schema and seed files to the team's GitHub repository.

Features implemented

Database Schema

Table Name	Description
users	Stores customer, admin, manager, and courier accounts with roles.
categories	Lookup table for menu categories (Burgers, Drinks, etc.).
menu_items	Menu entries with price, prep time, availability, and image URL.
orders	Customer orders with delivery address, status flow, and auto-total.
order_items	Line items linking orders to menu items with qty, price, subtotal.
payments	Payment records: method (cash/card/wallet), status, transaction ref.
invoices	Auto-generated invoice per order with unique number and total.

- Database Tables — Seven fully relational tables:
 - users — Stores customer, admin, manager, and courier accounts with role-based access.
 - categories — Lookup table for menu categories (e.g., مشاوي, حلويات, مشروبات).
 - menu_items — Menu entries with name, price, prep time, availability flag, and image URL.
 - orders — Customer orders with delivery address, status workflow, and auto-calculated total.
 - order_items — Line items linking orders to menu items with quantity, unit price, and subtotal.
 - payments — Payment records with method (cash/card/wallet/bank transfer), status, and transaction reference.
 - invoices — Auto-generated invoice per order with a unique invoice number and total amount.
- Automated Triggers:
 - trg_order_items_before_insert — Calculates subtotal = quantity × unit_price before each row insert.
 - trg_order_items_after_insert — Recalculates and updates the parent order's total_amount via SUM of all subtotals.
- Order Status Workflow: pending → confirmed → preparing → out_for_delivery → delivered / cancelled / refunded.
- Payment Status Workflow: pending → paid / failed / refunded.
- Referential Integrity: ON UPDATE CASCADE across all foreign keys; ON DELETE RESTRICT for critical entities; ON DELETE CASCADE for order_items.

Problems encountered

- Deciding whether order_items should allow quantity updates after insertion — the UNIQUE constraint on (order_id, menu_item_id) prevents duplicate rows for the same item.
- Selecting the correct numeric type for monetary columns — FLOAT was considered but ruled out due to floating-point precision errors.
- Determining the correct ON DELETE behavior for each foreign key relationship to balance data safety and application flexibility.
- Setting the right character encoding to support Arabic text in names and addresses.

Solutions applied

- Applied UNIQUE KEY (order_id, menu_item_id) on order_items so the application issues an UPDATE instead of a duplicate INSERT, keeping trigger logic simple and consistent.
- Used DECIMAL(10,2) for all price and amount columns to guarantee exact cent-level precision for all financial data.
- Used ON DELETE RESTRICT for parent tables (users, categories, menu_items, orders) to protect historical data, and ON DELETE CASCADE only for order_items whose rows are meaningless without their parent order.
- Set the database character set to utf8mb4 with utf8mb4_unicode_ci collation to fully support Arabic, emoji, and multilingual content.

Plan for next week

- Begin backend API development using Node.js / Express:
 - User authentication — register and login endpoints with bcrypt password hashing.
 - Menu browsing endpoint — list categories and items with filtering by availability.
 - Order placement endpoint — create order, add items, trigger auto-total calculation.
- Add server-side input validation to enforce business rules before data reaches the database.
- Write unit tests to verify trigger behavior and referential integrity constraints.
- Set up a GitHub Actions CI/CD pipeline to run schema and seed scripts automatically on push.
- Connect the backend to the existing database schema and test all endpoints with Postman.

Questions for instructor

- Should the invoices table store a per-item breakdown, or is referencing order_items via order_id sufficient for audit purposes?
- Is it acceptable to auto-generate the invoice_number in the application layer, or should a database stored procedure or sequence be used?
- For the courier role, should we create a separate deliveries table to track courier assignment and delivery timestamps, or is extending the orders table enough?